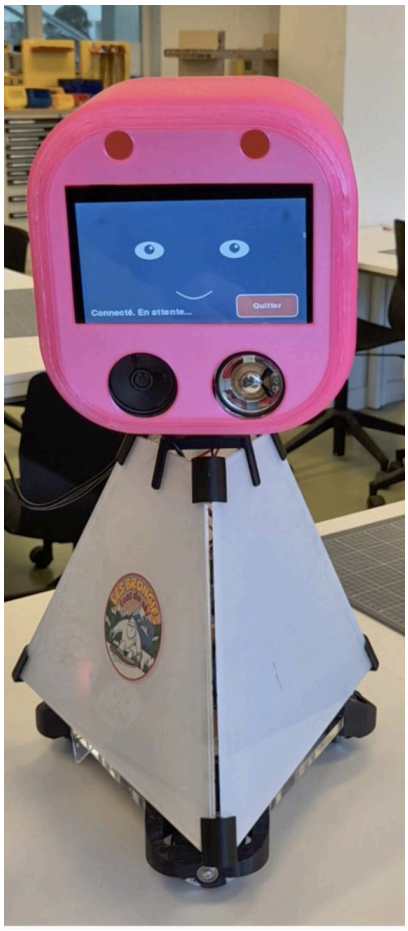
Project report: Empathetic Robot

Technical and functional summary

Raphaëlle POLAILLON
Adrien TESTON
Khalil BIBI
Armand CHOQUEL
Levani METREVELI

22 January 2026



Contents

1 Summary

2 Objectives

- 2.1 General objectives

3 Specifications

4 Mechanics and electronics

- 4.1 Wheeled base
- 4.2 Chassis and exoskeleton
- 4.3 Robot head: constraints and build
- 4.4 User interface: Raspberry Pi and peripherals
- 4.5 Head / body assembly
- 4.6 Power supply: portable battery

5 Software

- 5.1 Overview
- 5.2 Automatic speech recognition
- 5.3 Animated face and voice on the Raspberry Pi
- 5.4 PC → Raspberry Pi → Arduino communication

6 How to run the robot

7 Empathetic robot with a voice interface

- 7.1 Program description
- 7.2 User guide
- 7.3 AI models used
- 7.4 Customisation
- 7.5 Technical limitations
- 7.6 Modular system architecture

8 Results and demonstration

9 Possible improvements

10 Organisation and planning

11 Conclusion

1 Summary

Many children today experience social isolation, whether because of chronic illness, long hospital stays, anxiety disorders or difficulties related to bullying. This isolation can have a deep impact on their emotional well-being, their personal development and their sense of belonging.

In the face of these challenges, robotic technologies offer a new opportunity: designing empathetic companions able to provide emotional support, a reassuring presence and positive interactions. An empathetic robot can act as a mediator, a companion and a play partner, encouraging the expression of emotions, the maintenance of social ties and cognitive stimulation.

However, many robots on the market have limitations: interactions that are often pre-programmed, basic emotion recognition, a look and feel that is sometimes too “mechanical” or cold, and an underdeveloped educational dimension. Moreover, these robots often cost tens of thousands of euros, which makes them inaccessible for most families.

This is why we chose to develop a robot that stands out on two main axes. First, a reassuring and endearing physical presence: soft surfaces that are pleasant to the touch, to strengthen the feeling of comfort, calm and emotional attachment. Second, a richer educational and narrative dimension: beyond simple dialogue, the robot will be able to read stories expressively, support the child in their learning and offer suitable, playful educational content. These elements are the core motivation of our project: to create a robot that is not only empathetic and interactive, but also genuinely comforting, educational and affordable, in order to better meet the emotional and cognitive needs of children facing isolation.

2 Objectives

2.1 General objectives

The main functional objectives are:

- **Interaction:** hold a simple conversation with a user (speech recognition + response generation).
- **Mobility:** move on a flat surface and respond quickly to commands.
- **Empathy:** give the robot a “presence” (voice, face, simple gestures).
- **Demonstration:** integrate enough building blocks for a coherent presentation.

3 Specifications

The specifications set constraints on size, interaction, safety and autonomy. Table 1 summarises the main requirements.

Requirement	Quantification / criterion
Discreet and easy to move	Height < 60 cm; width/depth < 30 cm; weight < 4 kg
Simple conversation	Speech recognition; chatbot response time < 120 s
Playful content	Anecdotes; mini-games (e.g. songs)
Movement	Flat surface; must not tip over; response time < 3 s
Empathy	Facial expressions; simple movements
Safety	Non-hazardous materials; adjustable volume; no small dangerous parts
Autonomy	Rechargeable; target battery life of 20–30 minutes

Table 1 – Excerpts from the specifications

4 Mechanics and electronics

This part covers the wheeled base, the structure (chassis/exoskeleton), the head (support for the screen, speakers and microphone) and the power supply.

4.1 Wheeled base

The robot sits on a wheeled base recovered from a previous project. The base has 3 motors and three omnidirectional wheels. As one of the motors was poorly attached, the corresponding wheel behaves like a caster wheel and keeps the whole assembly stable. The two other (omnidirectional) wheels still allow movement in several directions to be controlled through the rotation speed of the motors. We managed to make the robot move forward, move backward, and turn left and right. We also implemented, on the Arduino board that drives the motors, a little dance for the robot, built from commands like the ones above.

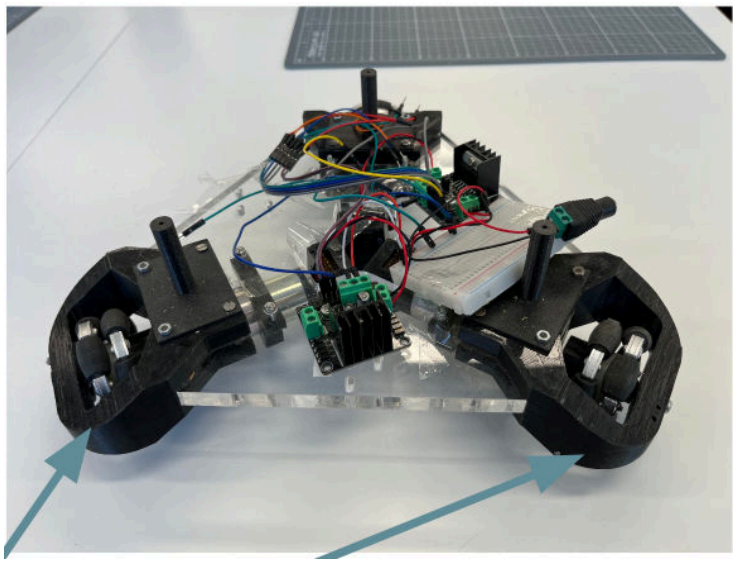


Figure 1 – Wheeled base and motors.

4.2 Chassis and exoskeleton

The chassis has two roles: (i) protecting the electronics and (ii) providing a stable and attractive structure. A wooden base, fixed above the motors and their Arduino board, protects the electronics of the base; an exoskeleton then covers the inside of the robot. The exoskeleton is made of three trapezoidal plates, cut with a laser cutter and held together by 3D-printed parts.



Figure 2 – Wooden base

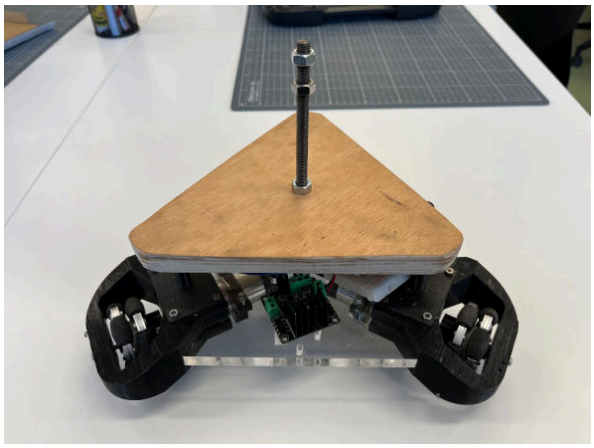


Figure 3 – With the protective base



Figure 4 – Chassis

4.3 Robot head: constraints and build

The head must:

- attach to the rest of the robot,
- house the screen, the speakers and the microphone,
- contribute to empathy (choice of colour and shape),
- be removable and adaptable for our V1.

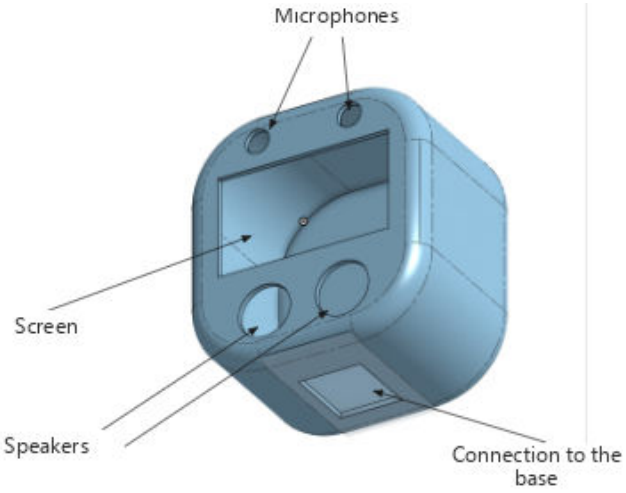


Figure 5 – Design of the robot's head

The physical build included adding spacers to hold the microphone, the screen and the speakers, as well as a support shaft to attach the robot's head to the rest of the chassis. The head was fixed to the chassis with a rod inserted both into this support shaft and into a 3D-printed part that fits onto the chassis.

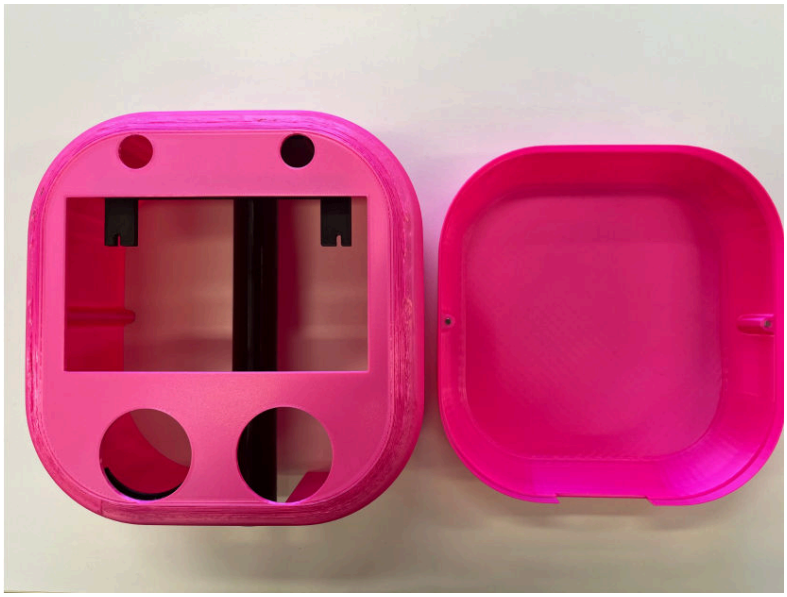


Figure 6 – Physical build of the head

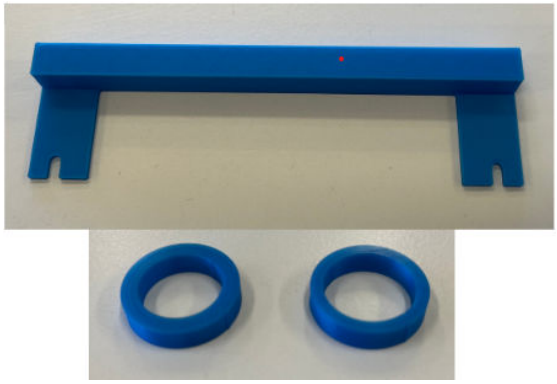


Figure 7 – Spacers for the screen (top) and the microphone (bottom)

4.4 User interface: Raspberry Pi and peripherals

The user interface is centred on a Raspberry Pi connected to:

1. a touchscreen,
2. an Arduino board, itself connected to the motors,
3. a USB microphone,
4. speakers, through an audio amplifier.

The screen is touch-enabled and connected to Wi-Fi; the Raspberry Pi is used to install the applications the robot needs.

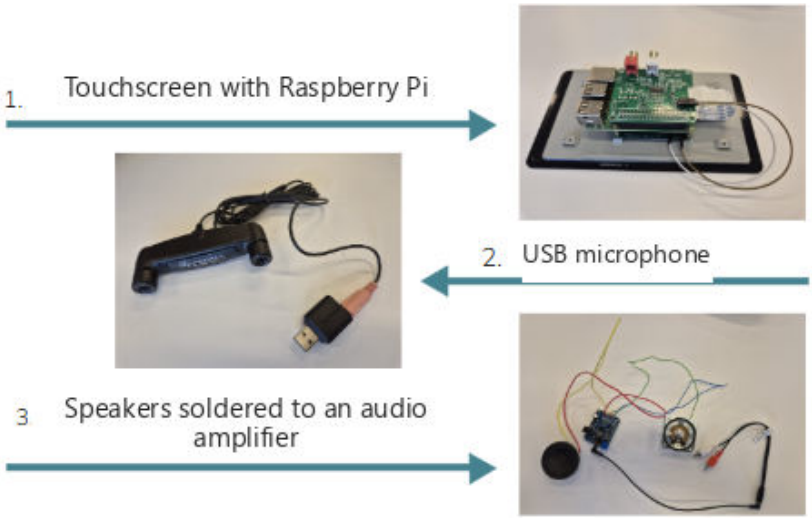


Figure 8 – User-interface components

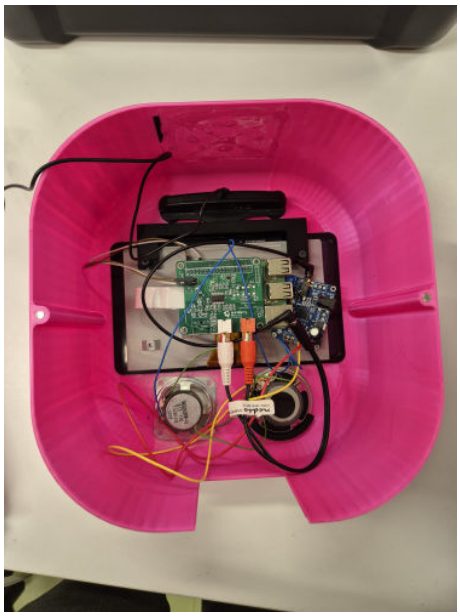


Figure 9 – Peripherals integrated into the head



Figure 10 – Peripherals integrated into the head

4.5 Head / body assembly

The head is mechanically attached to the body through a junction (connecting part) that ensures alignment and support (see the photo on the title page).



Figure 11 – Part fixing the chassis to the head

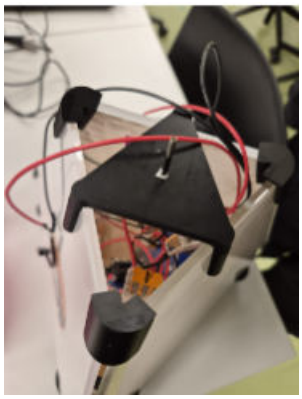


Figure 12 – Part fixing the chassis to the head

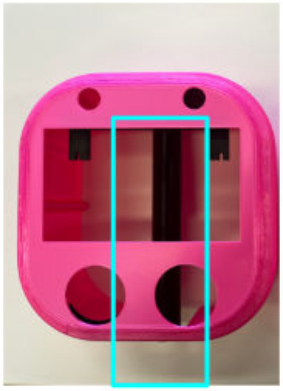


Figure 13 – Part fixing the head to the chassis

4.6 Power supply: portable battery

Four subsystems are powered:

- screen + Raspberry Pi: 5 V
- audio amplifier: 5 V
- motors: 10 V
- Arduino: 5 V

A USB-C rechargeable battery was integrated, with distribution of the voltages required by all the components.

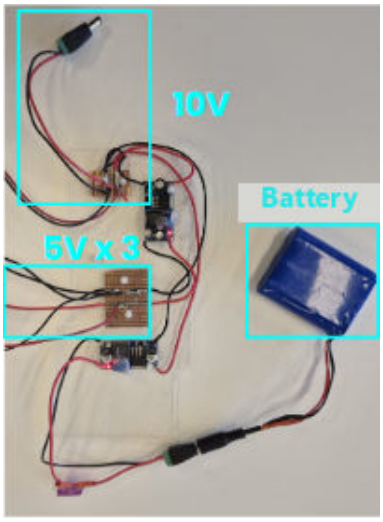


Figure 14 – Battery and voltage distribution

5 Software

The software is built around four building blocks: speech recognition, the chatbot, the face + voice interface on the Raspberry Pi, and the PC → Raspberry Pi → Arduino communication chain.

5.1 Overview

The main tasks coded and integrated are:

- Speech recognition (audio → text) with French/English translation, since the training data of our models is in English.
- Chatbot: analysis of the request and generation of responses.
- Raspberry Pi: animated face + speech synthesis + audio playback.
- Command handling: “move forward”, “move back”, “dance”, “sing”.

5.2 Automatic speech recognition

A transcription demonstration was carried out from an audio file (a French children's recording from the audio-lingua.ac-versailles.fr site). The system produces a transcription in French and then a translation into English. We used Whisper's “medium” model, which combines robustness and speed, giving an execution time of about 3 seconds for recognition + translation (under test conditions) for a 4-second audio clip.

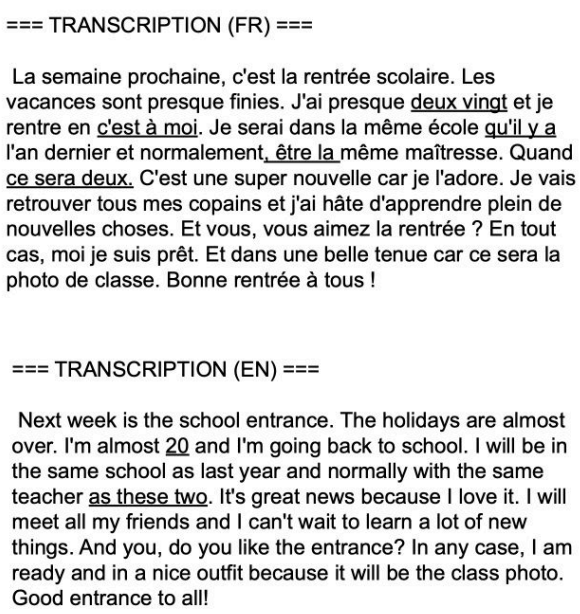


Figure 15 – Example of transcription (French input, English translation).

5.3 Animated face and voice on the Raspberry Pi

Goal: give the robot a presence.

- **Display:** a full-screen interface (Pygame) shows a face (eyes + mouth).
- **Speech synthesis:** the received text is turned into audio (MP3) with gTTS (Google voice) and then played.
- **Animation:** while the audio plays, the mouth opens and closes to simulate speech, and the eyes blink to make the face look more natural.
- **Clean exit:** a “Quit” button closes the program properly.



Figure 16 – The robot's face when it is active (on-screen text: “Connected. Waiting…”, “Quit”).

5.4 PC → Raspberry Pi → Arduino communication

The communication chain implements simple and robust control:

1. **PC:** sends a text message (command or sentence) over WebSocket.
2. **Raspberry Pi:** receives the text, displays it and synthesises speech; detects keywords.
3. **Arduino:** receives a short command over USB (e.g. AVANCE, RECULE, DANSE, CHANTE: forward, back, dance, sing).
4. **Motors:** execute the command (e.g. move forward for 3 seconds, then stop).

On the Raspberry Pi side, we implemented simple keyword detection:

- if the text contains “avance” (forward) → send AVANCE to the Arduino;
- if the text contains “recule” (back) → send RECULE;
- if the text contains “danse” (dance) → send DANSE;
- if the text contains “chante” (sing), the Raspberry Pi plays a song stored locally;
- otherwise → handle it as a normal conversation (chatbot + voice + face).

This approach is deliberately simple to guarantee a working prototype, and can be improved.

6 How to run the robot

To run the robot, first make sure that the Raspberry Pi (via the screen) and the PC are connected to the same Wi-Fi network. This is essential for network communication between the two

machines.

Then, on the Raspberry Pi, edit the file `arduino.py` to set the PC's IP address: replace the default IP address with the address of the PC that will act as the server. This step allows the Raspberry Pi program to connect to the service started on the PC.

On the PC, start the local server by running `pc_serveur.py` (provided in the deliverables). Once the server is running, the Raspberry Pi can connect to it to receive messages and commands.

Finally, on the Raspberry Pi screen, start the main program `arduino.py` from the terminal: go to the project folder, activate the Python virtual environment and run the script:

```
cd ~/robot_stt
source .venv/bin/activate
python arduino.py
```

Once both programs (PC server and Raspberry Pi client) are running, commands can be given directly from the PC terminal. They are sent to the robot through the server, then passed on to the Raspberry Pi and the Arduino to trigger the actions (movement, singing, etc.).

The chatbot interaction could not be integrated into the robot itself, because the Raspberry Pi 3 cannot deliver the power needed to run the models locally. We therefore decided to run the AI on an external PC. One improvement would be to run the AI locally in the robot by replacing the Raspberry Pi 3 with a more powerful Raspberry Pi 5.

7 Empathetic robot with a voice interface

7.1 Program description

The program `robot_with_voice.py` is an empathetic conversational chatbot with an animated graphical interface and full voice capabilities. It combines several artificial-intelligence technologies to create a natural interactive experience.

7.1.1 Main features

- **Animated graphical interface:** a robot face with expressive eyes that adapt to the detected emotions (joy, sadness, anger, surprise, confusion, tiredness).
- **Speech recognition:** automatic transcription of speech into text with Whisper (OpenAI).
- **Smart response generation:** several candidate responses generated by models.
- **Neural reranking:** selection of the best response among the candidates with a Cross-Encoder.
- **Speech synthesis:** responses are spoken in French with Google TTS (gTTS).
- **Emotion detection:** analysis of the user's emotional state and adaptation of the robot's face.
- **Automatic translation:** conversion of the English responses into French.

7.2 User guide

7.2.1 Prerequisites

- Python 3.8+
- Dependencies: pygame, gtts, torch, transformers, openai-whisper
- Models pre-downloaded into `local_models/` (about 5 GB)
- A working microphone

7.2.2 Starting the program

Loading takes 1–3 minutes at first start-up, because all the models and libraries are loaded.

7.2.3 Interaction

1. **Wait for the "Ready" message:** the interface shows a happy face.
2. **Press SPACE:** starts a 4-second voice recording.
3. **Speak clearly:** say a sentence in French or English.
4. **Wait:** the system transcribes, analyses and generates a response (5–15 seconds).
5. **Listen to the response:** the robot shows the appropriate emotion and speaks its answer.
6. **Press Q:** quit the program.

7.2.4 Example interactions

User	Robot (Denise)	Emotion
"What's your name?"	"My name is Denise! Nice to meet you."	Joy
"I'm sad today"	"I'm sorry to hear that. Do you want to talk about it?"	Sadness
"Tell me a joke"	"Why do divers always fall backwards off the boat? Because otherwise they'd fall into the boat!"	Joy
"I don't understand"	"No problem, I can rephrase..."	Confusion

Table 2 – Example interactions with the robot (the robot speaks French; translated here)

7.3 AI models used

Model	Use	Size
Whisper (medium)	Audio transcription	1.5 GB
BERT-base-NER	Named-entity recognition	420 MB
BlenderBot-1B-distill	Conversational dialogue	1.3 GB
BlenderBot-empathetic	Empathetic responses	1.3 GB
Cross-Encoder MS-MARCO	Response reranking	90 MB
DeBERTa-v3 MNLI	Intent classification	350 MB
Opus-MT-en-fr	EN → FR translation	300 MB

Table 3 – AI models used

7.4 Customisation

7.4.1 Emotion handling

Emotion detection is done with the DeBERTa model, which also chooses which model will be used to generate the candidate responses.

7.5 Technical limitations

- **Response time:** 100–120 seconds depending on the processor's power.
- **Audio quality:** requires a quiet environment and a microphone close to the source for good transcription.
- **RAM:** at least 8 GB recommended, ideally more.
- **Limited context:** the system only remembers the current conversation.

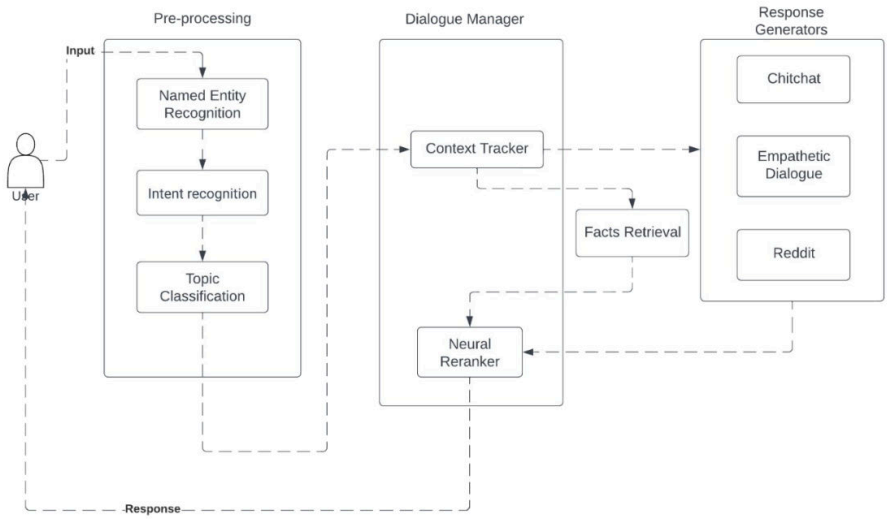


Figure 17 – Chatbot architecture diagram

7.6 Modular system architecture

Our implementation follows a modular architecture in three distinct layers, shown in the figure above, which clearly separates the processing responsibilities.

7.6.1 Pre-processing layer

After the user speaks into the microphone and the speech-recognition algorithm turns the audio into text, this text is translated and then sent to the first layer of the architecture. The first layer has three essential modules: named-entity recognition (NER) with BERT-base-NER, which identifies people, places and organisations in the user's input; intent recognition, implemented with DeBERTa-v3-MNLI, which classifies the utterance according to its communicative goal (chitchat, emotional support, question, greeting, goodbye); and topic classification, which categorises the subject of the conversation.

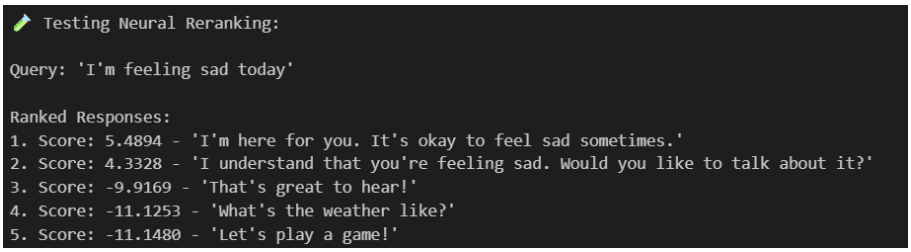


Figure 18 – Example test of the neural reranker

7.6.2 Dialogue-manager layer

The second layer orchestrates the conversational flow through a Context Tracker, which keeps the conversation history in a Python list, and a Facts Retrieval module, which extracts relevant information from the context.

The central component is the Neural Reranker, based on the MS-MARCO Cross-Encoder, which scores and ranks the candidate responses according to their semantic relevance to the user's input.

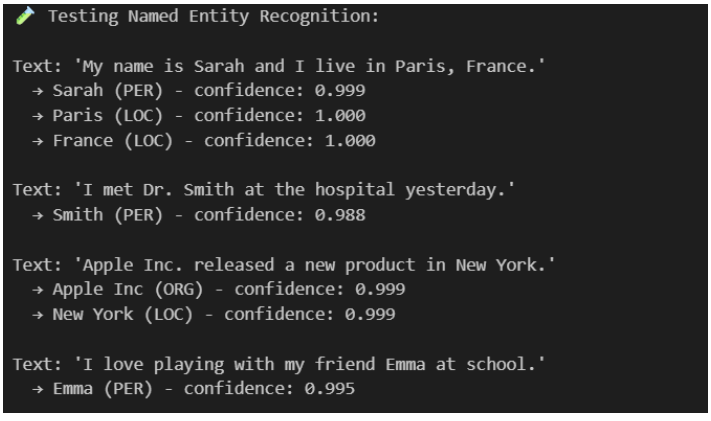


Figure 19 – Example tests of the NER

7.6.3 Response-generation layer

The third layer produces several diverse candidate responses, depending on the model chosen by the intent classifier: chitchat responses with BlenderBot-1B-distill for light conversation, or

empathetic dialogue responses with BlenderBot-1B-distill-empathetic for emotional support. The Neural Reranker then selects the most relevant response among the candidates generated by this layer and returns it to the user. As this response is generated in English, it is translated into French before being spoken.

7.6.4 Rationale for the chosen architecture

We deliberately chose this modular architecture rather than a single, simplistic open-source conversational model (such as DialoGPT alone or a fine-tuned GPT-2), for several strategic reasons.

First, this approach offers considerable room to evolve: each module can be replaced, improved or extended independently without redesigning the whole system. For example, we could easily add a new response generator specialised in factual questions, or replace the reranker with a better model, without touching the other components.

Second, generating multiple candidates followed by smart reranking guarantees, in the long run, a better response quality for our task than a complete open-source model.

Third, this architecture makes debugging and maintenance easier: when a response is inadequate (too political, off-topic, inappropriate for the context, or simply wrong, for example), we can identify precisely which module is at fault (wrong intent detection, failing reranking, or unsuitable generator).

7.6.5 Future developments

In the long term, this modularity makes ambitious extensions possible, such as adding an information-retrieval system (RAG) for factual questions, integrating a personality model to adapt the tone of the responses, or using reinforcement learning to fine-tune the reranker on real user feedback.

Although this architecture is more complex to implement at first than a simple call to a single model, it is a robust and scalable foundation that anticipates future needs for improvement and adaptation, avoiding the constraints of a monolithic model that is hard to evolve.

8 Results and demonstration

The prototype achieves a coherent integration:

- a physically assembled robot (base + chassis + head) with a Raspberry Pi interface,
- a basic voice interaction (transcription / synthesis),
- an animated face synchronised with speech,
- a text command chain PC → Raspberry Pi → Arduino to trigger actions.

9 Possible improvements

The improvement areas we identified are:

- **Autonomy:** make the screen and the interface fully autonomous and robust on battery; host the artificial intelligence and speech recognition on the Raspberry Pi.
- **Embedded AI:** improve the artificial intelligence, run more of it directly on the robot, and improve the response time.
- **Aesthetics / empathy:** improve the look (colours, shapes) and enrich the facial expressions.
- **Safety:** strengthen the mechanical and electrical protections (visible cables and unsecured detachable parts).
- **Performance:** reduce the overall response time (ASR + chatbot + TTS).

10 Organisation and planning

A macro-planning was drawn up to structure the project over 16 sessions: scoping, technical design, getting started with the hardware, Raspberry Pi/Arduino integration, trajectories, AI, testing and iterations.

More generally, we split the work within the group, mainly between the mechanics/electronics part and the software/AI part. We worked in parallel to be more productive, then brought our respective work together to try to make the robot work as a whole. Finally, we worked together on assembling the different parts of the robot and on adapting to unforeseen issues (AI too heavy for the Raspberry Pi, or missing spacers for the speakers, for example).

Session	Tasks / deliverables	Owner(s)
1	Scoping, specifications, team rules.	Everyone
2	Functional design and technical choices.	Everyone
3–4	Parallel prototyping: getting started with the Arduino [1], getting started with the Raspberry Pi [2], planning a strategy for Python/the chatbot [3].	[1] Armand & Raphaëlle; [2] Adrien; [3] Levani & Khalil
5	Getting started with the motors [1]; Raspberry Pi–screen link [2]; testing the first chatbot prompts [3].	[1] Armand & Raphaëlle; [2] Adrien; [3] Levani & Khalil
6	Arduino–motor link [1], getting started with the speakers [2], improving the chatbot [3].	[1] Armand & Raphaëlle; [2] Adrien; [3] Levani & Khalil
7–8	Trajectory control [1], animation effects [2], improving the chatbot [3].	[1] Armand & Raphaëlle; [2] Adrien; [3] Levani & Khalil
9–12	Building the overall chassis and integrating the screen, speakers, chatbot, etc. into the chassis.	Everyone
13	Integrating narrative content (songs, stories…) [1], mechanical safety (rounded corners, fixings) [2].	[1] Levani & Khalil; [2] Armand & Raphaëlle & Adrien
14	Final integration, last improvements.	Everyone
15	Bug testing 1.	Everyone
16	Bug testing 2.	Everyone

Figure 20 – Detailed macro-planning (16 sessions)

11 Conclusion

This project aimed to design an empathetic and affordable robot, able to provide a reassuring and interactive presence for children experiencing social isolation. The initial objectives were broadly met: the robot can move stably on flat ground, interact vocally with a user, and express a minimal form of empathy through an animated face, speech synthesis and simple behaviours. Despite some technical limitations, notably related to the performance of the Raspberry Pi 3 for running heavy artificial-intelligence models locally, the final prototype is a coherent and working proof of concept.

Beyond the technical result, this project allowed us to develop many transferable skills. On the technical side, we deepened our knowledge of mobile robotics, hardware–software integration, inter-system communication and applied artificial intelligence (speech recognition, natural language processing, speech synthesis, modular chatbot architecture). On the methodological side, we learned to work under strong time constraints, to prioritise the essential features, to iterate quickly and to make realistic trade-offs between ambition and feasibility.

Teamwork was also a key lesson: it highlighted the importance of communication, coordination and flexibility in a multidisciplinary project.

Finally, this project opens up many avenues for improvement, both technical and human (the robot's expressiveness, personalisation, richer educational content). It is therefore a solid basis for future developments, and a concrete illustration of how robotics and artificial intelligence can be put at the service of well-being and human support.